

컴퓨터비전 중간고사 정리

이 과목을 한 줄로

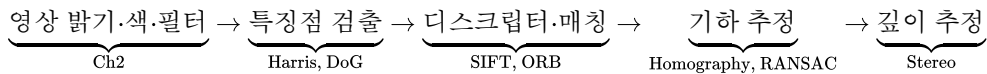
"영상에서 의미 있는 점을 찾고, 그 점들의 관계로 장면의 구조와 깊이를 복원한다"
2D 처리 → 특징 추출 → 매칭 → 기하 추정 → 깊이 추정의 한 흐름으로 묶어서 외워라.

0. 한눈에 보는 시험 핵심 지도

시험장에서 즉시 떠올라야 할 7가지

1. 코너 = 여러 방향에서 밝기 변화가 큰 점
2. 좋은 특징의 3조건 = 불변성 · 변별성 · 재현성
3. Homography = 3×3 행렬, 최소 4쌍 대응점 (자유도 8 ÷ 식 2 = 4)
4. Least Squares는 outlier에 약함 · RANSAC은 강함
5. 핀홀 투영: $x = \frac{fX}{Z} + p_x, y = \frac{fY}{Z} + p_y$
6. 스테레오 깊이: $Z = \frac{fB}{d}$ (disparity에 반비례)
7. 실습 키워드: LUT 감마 · YUV의 Y · Gaussian → Sobel/Canny · Harris · SIFT/ORB · PSNR/SSIM

전체 파이프라인



원본 슬라이드 링크

[Ch1 Overview](#) · [Ch2 2D Processing](#) · [Ch3 2D Processing II](#) · [Ch4 Feature](#) · [Ch5 Stereo](#)

0-α. 비전공자용 용어 미니 사전

이 표만 알면 본문이 90% 읽힌다 — 먼저 이 표를 숙지하고 본문으로 내려가자

용어	한 줄 정의	일상 비유
픽셀 pixel	영상을 이루는 최소 점 (보통 밝기 0 ~ 255)	모자이크 타일 한 조각
채널 channel	한 색 성분의 2D 맵 (R·G·B 등 3장)	빨강·초록·파랑 필터로 각각 찍은 흑백 사진 3장
히스토그램 histogram	픽셀 값이 몇 개씩 있는지 세어 그린 막대그래프	학급 성적 분포표
커널 kernel	이미지를 훑고 다니며 곱해주는 작은 숫자판 (보통 3 × 3)	도장·스탬프
컨볼루션 convolution	커널을 이미지 위로 미끄러뜨려 새 이미지를 만드는 연산	사진 전체에 도장을 한 칸씩 찍어가며 효과 입히기
필터링 filtering	컨볼루션으로 블러·샤프닝·엣지 검출 등을 만드는 행위	인스타 필터 하나 입히는 것
에지 edge	밝기가 급격히 변하는 경계선	물체의 윤곽선

용어	한 줄 정의	일상 비유
코너 corner	두 방향 이상의 에지가 만나는 점	방의 모서리
특징점 feature point	사진이 바뀌어도 다시 찾아낼 수 있는 눈에 띄는 점	얼굴의 점·눈썹 끝 같은 랜드마크
디스크립터 descriptor	특징점 주변을 요약한 숫자 벡터	지문·바코드
매칭 matching	두 사진의 디스크립터끼리 짝짓기	지문 대조
호모그래피 homography	같은 평면을 다른 각도에서 찍었을 때의 변환 공식 (3×3 행렬)	종이를 기울여 찍었을 때 어떻게 찌그러져 보이는지를 알려주는 표
outlier (이상치) / inlier (정상치)	잘못된 점 / 모델과 잘 맞는 점	반 평균을 망치는 한두 명 / 나머지 정직한 학생들
RANSAC	일부로 가설 세워보고 지지자(inlier)가 가장 많은 가설 채택	여러 후보 중 투표로 우승자 뽑기
disparity (시차)	좌·우 두 카메라에서 같은 점의 x 좌표 차이	왼쪽 눈·오른쪽 눈 번갈아 감을 때 손가락이 얼마나 움직여 보이는지
PSNR / SSIM	복원 이미지가 원본과 얼마나 비슷한지 재는 두 가지 점수	"숫자 오차로 채점" vs "사람 눈 기준 채점"

📖 읽는 순서 팁

1. 이 사전을 먼저 훑는다 → 2) §0의 "7가지"를 빠르게 본다 → 3) §1부터 순서대로. 수식이 나오면 먼저 그 위·아래 글로 된 설명부터 읽고 수식은 마지막에.

0-β. 컴퓨터비전이란? (Computer Vision Definition)

📖 공식 정의

컴퓨터비전 = "디지털 영상/비디오에서 의미 있는 정보를 자동으로 추출하고 이해하는 학문 및 기술"

- 입력: 2D 영상(정지/동영상) / 출력: 기하 구조, 객체 라벨, 깊이, 자세 등 고차원 의미
- 인간 시각 체계를 컴퓨터로 구현하려는 시도 — "컴퓨터가 본다"

인접 분야와의 구분

분야	입력	출력	목적
영상 처리 (Image Processing)	영상	영상	개선·변환 (블러·샤프닝)
컴퓨터비전 (Computer Vision)	영상	의미·구조	이해·인식·복원
컴퓨터그래픽스 (Computer Graphics)	모델/파라미터	영상	생성·렌더링
패턴 인식 (Pattern Recognition)	특징 벡터	클래스	분류·군집

대표 응용

- 자율주행: 차선/보행자/신호등 검출, SLAM, 3D 재구성
- 의료영상: 종양 분할, 병변 검출
- AR/VR: 평면 인식, 카메라 추적, 3D 가상객체 배치
- 로봇: 물체 잡기(grasping), 환경 인식
- 감시·보안: 얼굴 인식, 이상행동 탐지

- 콘텐츠: 파노라마, 포토스티칭, 이미지 검색

1. 영상 구조의 기초 어휘 — 왜 '코너'가 중요한가

1-0. 영상 = 2D 신호 (디지털 이미지의 본질)

한마디로 — "이미지는 격자 위 숫자판"

- 1D 신호: $f(x)$ — 시간에 따른 음성, 주가 그래프처럼 값 하나가 위치에 대응
- 2D 신호: $f(x, y)$ — 격자(grid) 위 밝기값 — 디지털 이미지가 바로 이것
- 컬러 이미지는 채널마다 2D 신호 3장: $f_R(x, y), f_G(x, y), f_B(x, y)$
- 수학적 연산(미분·컨볼루션·푸리에변환)을 2D로 확장하면 모두 영상처리 도구로 변신

구분	표현	예
1D 신호	$f(x)$	음성, ECG, 주가
2D 신호	$f(x, y)$	흑백 이미지 (H×W 픽셀)
2D 멀티채널	$f(x, y, c)$	컬러 이미지 (H×W×3)

- **Feature map** = CNN이 계산한 중간 2D 신호 (해석: "각 위치에서 특정 패턴이 얼마나 강한가")

1-1. 핵심 어휘 4개

어휘	정의	매칭 유리도
에지(edge)	밝기가 급격히 변하는 경계선	△ (한 방향만 강함)
코너(corner)	두 에지 이상이 만나는 점	⊙ (여러 방향 강함)
텍스처(texture)	반복 패턴·미세 질감	○
descriptor	특징점 주변을 요약한 숫자 벡터	— (매칭 단위)

왜 코너가 에지보다 매칭에 유리한가

- 에지는 선을 따라 움직여도 비슷해 보임 → 위치 모호
- 코너는 어느 방향으로 움직여도 달라짐 → "이 점이 바로 이 점"이라고 식별 가능

1-2. 좋은 특징의 3조건 (시험 단골)

1. 불변성 **Invariance** — 스케일·회전·조명·시점 변화에 값이 크게 흔들리지 않음
2. 변별성 **Distinctiveness** — 다른 점과 확실히 구분됨
3. 재현성 **Repeatability** — 다른 사진에서도 같은 위치 근처에서 다시 검출됨

1-3. 특징 추출 2단계 파이프라인

모든 특징 기반 알고리즘(Harris/SIFT/ORB)은 2단계 구조

1. **Detection (검출)** — "어디가 특징점인가?" → 위치 찾기
 - Harris: 코너 응답 R, SIFT: DoG 극값, ORB: FAST 코너
2. **Description (기술)** — "그 점 주변이 어떻게 생겼는가?" → 숫자 벡터로 요약
 - SIFT: 128차원, ORB: 256비트 이진 벡터

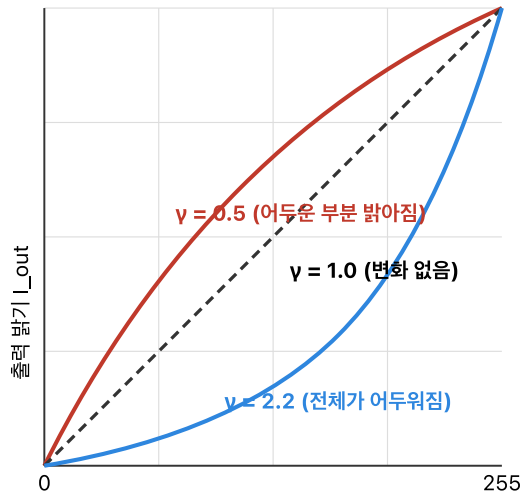
- 매칭(Matching)은 **Description** 벡터끼리 거리 비교로 이뤄짐

2. 2D 영상 처리 — 밝기·색·필터링

2-1. 감마 보정

$$\text{감마 보정: } I_{\text{out}} = 255 \cdot (I_{\text{in}}/255)^\gamma$$

세 곡선은 같은 사진을 γ 값만 바꿔 돌렸을 때의 입출력 관계. 가로축=원본 밝기, 세로축=보정 후 밝기.



한 줄 암기: $\gamma < 1$ 밝게, $\gamma > 1$ 어둡게. 사람 눈은 어두운 변화에 민감하고 모니터는 $\gamma \approx 2.2$.

시각 효과 (같은 원본에 γ 적용)

원본 ($\gamma=1$)

$\gamma=0.5$ — 어두운 중간값이 밝아져 노출이 올라간 효과

$\gamma=2.2$ — 중간값이 내려가 대비가 강해지고 어두워짐

시험 포인트:

- $\gamma < 1 \rightarrow$ 어두운 부분 $\uparrow$ (사진 밝게)
- $\gamma > 1 \rightarrow$ 전체가 어두워짐
- 모니터가 $\gamma \approx 2.2$ 특성 $\rightarrow$ 저장 시 보통 역감마($1/2.2$) 보정
- OpenCV 실습에서는 cv2.LUT로 룩업 테이블을 미리 만든 뒤 적용 (공간-시간 $\uparrow$)

한마디로 — "밝기를 비선형으로 휘게 하는 손잡이"

- 사람 눈은 어두운 곳의 차이는 예민하게, 밝은 곳의 차이는 둔하게 느낀다.
- 그래서 밝기 값을 그냥 2배 해버리면 어두운 쪽이 너무 뭉개지거나 날아가 보인다.
- 감마 보정은 0 ~ 255 값을 **구부러진 곡선으로 다시 매핑**해서 눈에 자연스럽게 맞춘다.

$$I_{\text{out}} = 255 \left(\frac{I_{\text{in}}}{255} \right)^\gamma$$

수식 풀이 (수식 공포 금지)

- $I_{\text{in}}/255$: 밝기를 0 ~ 1로 정규화
 - $(\cdot)^\gamma$: γ 제곱 $\rightarrow \gamma$ 가 작으면 값이 커지고, 크면 작아짐
 - $\times 255$: 다시 0 ~ 255로 복원
- 즉 "정규화 $\rightarrow$ 구부리기 $\rightarrow$ 복원"의 세 단계.

γ	효과	왜	일상 비유
$\gamma < 1$	어두운 부분이 밝아짐	지수가 작을수록 출력이 커짐	어두운 방에 조명을 살짝 켜 느낌
$\gamma > 1$	전체적으로 어두워짐	지수가 클수록 출력이 작아짐	선글라스를 한 겹 씌운 느낌

실습 포인트 — Gamma_Correction.ipynb

- cv2.LUT — LUT(Look-Up Table)는 미리 계산해 둔 표

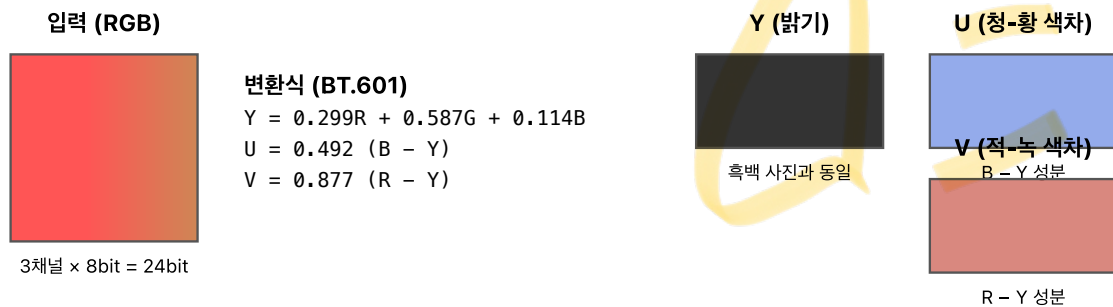
- "공간을 써서 시간을 줄인다"는 전형적 최적화 기법

🔗 CRT 모니터의 $\gamma \approx 2.2$ — 역사적 배경 (시험 단골)

- CRT(브라운관) 모니터는 입력 전압과 출력 밝기 관계가 비선형: $I_{display} \approx V_{in}^{2.2}$
- 이를 보정하려면 이미지를 저장할 때 미리 $1/2.2$ 제곱으로 왜곡해 두어야 함
- 최종: $(I^{1/2.2})^{2.2} \approx I \rightarrow$ 눈에 선형으로 보임
- 현대 LCD/OLED도 호환성 위해 sRGB 색공간에서 $\gamma \approx 2.2$ 유지
- 사람 눈은 어두운 영역 변화에 민감 $\rightarrow \gamma$ 보정이 지각적으로 자연스러운 이유

2-2. RGB $\leftrightarrow$ YUV 색공간

YUV 색 공간: 밝기(Y)와 색(U,V)을 분리해서 쓰는 이유



서브샘플링 (사람 눈의 특성 활용)

사람 눈은 밝기(Y)에 민감, 색(U,V)에 덜 민감 $\rightarrow$ UV만 해상도를 줄여도 체감 품질 거의 동일.

4:4:4 (손실 없음)



Y와 UV 해상도 동일
 $\rightarrow$ 스튜디오/편집

4:2:2 (U,V 수평 1/2)



UV 수평 해상도 절반
 $\rightarrow$ 방송 송출

4:2:0 (가장 많이 사용)



UV 수평·수직 1/2 $\rightarrow$ 1/4
 $\rightarrow$ JPEG, MPEG, H.264

압축 효과

4:4:4 $\rightarrow$ 원본 대비 100%

4:2:2 $\rightarrow$ 2/3 데이터

4:2:0 $\rightarrow$ 1/2 데이터

거의 같아 보이면서 절반!

시험: "왜 YUV를 쓰나?" $\rightarrow$ ① 밝기/색 분리 ② 색채널만 다운샘플링 가능 ③ 압축률 $\uparrow$ 체감화질 $\downarrow$ 적음. `cv2.cvtColor(img, cv2.COLOR_BGR2YUV)`

🔗 한마디로 — "밝기(Y)와 색(U,V)을 분리해서 따로 만진다"

- RGB는 빨강·초록·파랑의 세 조명을 섞어 색을 만드는 방식
- YUV는 "얼마나 밝은가(Y) + 어떤 색인가(U,V)" 로 나눠 보는 방식
- 비유: 흑백 사진 한 장(Y) + 색 레이어 두 장(U,V)을 포개 놓은 것

- Y: 밝기(Luminance) — 흑백사진이 Y만 쓴 것
- U, V: 색차(Chrominance)

🔗 왜 Y 채널만 조작하는가

대비를 높일 때 RGB를 직접 건드리면 색이 왜곡됨.

밝기 정보 Y만 평활화 $\rightarrow$ 색감 보존

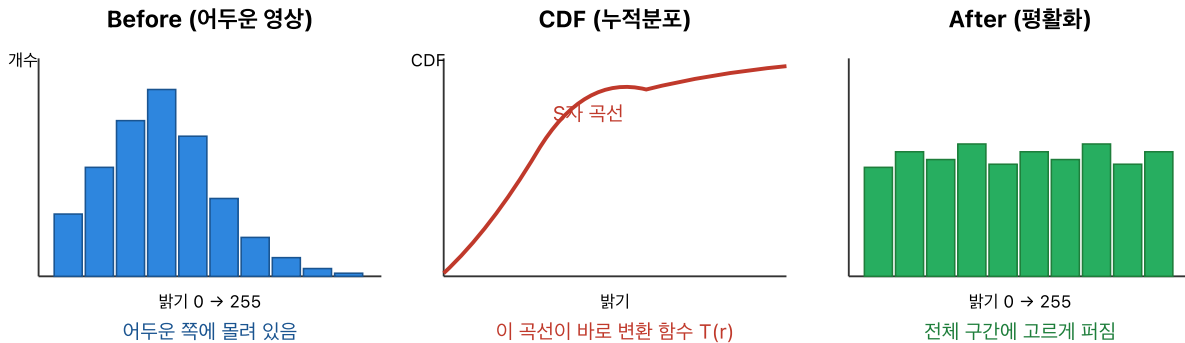
비유: 사진을 선명하게 만들 때 명도(밝기)만 조절하고 채도(색)는 건드리지 않는 포토샵 팁과 같다.

- `cv2.cvtColor` 로 BGR↔YUV, `cv2.split` 으로 분리
- OpenCV 기본 채널 순서는 RGB가 아니라 **BGR**

2-3. 히스토그램 평활화

히스토그램 평활화: 한쪽에 몰린 밝기를 골고루 퍼기

핵심 공식: $s = T(r) = (L-1) \cdot \text{CDF}(r)$ — 누적분포함수를 그대로 매핑 함수로 쓴다.



절차 (3단계)

- ① 히스토그램: 각 밝기값(0~255)의 픽셀 개수 $h(r)$ 세기
- ② CDF 계산: 누적합 $\text{cdf}(r) = \sum_{i=0..r} h(i)$ 그리고 정규화
- ③ 매핑: $s = \text{round}((L-1) \cdot \text{cdf}(r) / N)$ — N 은 총 픽셀 수, $L=256$

시험 포인트: "CDF를 매핑함수로 쓰면 출력 히스토그램이 이론적으로 균등분포에 근접" — OpenCV: `cv2.equalizeHist`

한마디로 — "몰려 있는 밝기 값을 골고루 퍼서 대비를 살린다"

- 히스토그램 = 밝기 값의 분포표. 대부분의 픽셀이 중간 밝기에 몰려 있으면 사진이 흐릿해 보인다.
- 평활화는 "성적이 60~70점에 쏠려 있을 때 40점과 90점을 억지로 만들어 분포를 넓히는 것"과 비슷
- 누적분포함수(CDF)란? "이 밝기 이하인 픽셀이 몇 %인가" 그래프 — 이 비율을 그대로 새 밝기 값으로 쓴다.

- 목적: 명암 분포를 고르게 퍼서 대비 향상
- 컬러 처리: YUV 변환 → Y에 `equalizeHist` → merge
- 누적분포함수(CDF)를 이용해 원래 몰려 있던 값들을 펼침

2-4. 컨볼루션 필터링

컨볼루션 필터링: 3×3 커널을 영상 위에서 미끄러뜨리기

규칙: 창 안 9개 픽셀 × 커널 9개 값을 모두 곱해 더한 값이 결과 영상의 한 픽셀이 된다.

원본 영상 f

10	20	30	40	50
12	22	32	42	52
14	24	34	44	54
16	26	36	46	56
18	28	38	48	58

노란 창(3×3)이 현재 계산 중

커널 h (블러)

1	2	1
2	4	2
1	2	1

합 = 16 → 1/16로 정규화

중앙 픽셀 계산 (점곱 후 정규화)

$$\begin{aligned}(22 \cdot 1 + 32 \cdot 2 + 42 \cdot 1) &= 22 + 64 + 42 = 128 \\ (24 \cdot 2 + 34 \cdot 4 + 44 \cdot 2) &= 48 + 136 + 88 = 272 \\ (26 \cdot 1 + 36 \cdot 2 + 46 \cdot 1) &= 26 + 72 + 46 = 144\end{aligned}$$

$$\text{합: } 128 + 272 + 144 = 544$$

$$\text{결과: } 544 / 16 = 34$$

34

← 결과 영상 g 의 (2,2)

다음 단계: 창을 오른쪽으로 한 칸 밀고 반복.

경계 픽셀은 zero-padding, replicate 등으로 처리.

커널만 바꾸면 블러(평균/가우시안), 샤프닝, 엠보싱, 에지(Sobel) 등 다양한 효과가 같은 틀에서 나온다.

한마디로 — "작은 도장을 사진 위에 한 칸씩 찍어가며 효과를 입힌다"

- 커널(작은 숫자판, 보통 3×3)을 이미지 왼쪽 위부터 오른쪽 아래까지 한 픽셀씩 밀면서
- 그 자리에서 곱친 숫자끼리 곱해 합산해 새 픽셀 값을 만든다.
- 커널 숫자만 바꾸면 블러·샤프닝·엣지 검출·엠보싱이 전부 같은 방식으로 나온다.

$$g(x, y) = \sum_{i=-k}^k \sum_{j=-l}^l h(i, j) f(x - i, y - j)$$

수식 풀이

- $\sum \sum$: "커널 범위 안 모든 칸을 돌며" 라는 뜻
- $h(i, j) f(x - i, y - j)$: "커널 값 × 곱친 이미지 값"
- 전체 합이 새 픽셀 $g(x, y)$ — 즉 가중 평균

- f : 원본, h : 커널, g : 결과
- 엠보싱 · 샤프닝 · 평균 블러 — 모두 "커널이 다를 뿐 같은 틀"

커널 느낌	하는 일	일상 비유
모두 양수·합=1	주변을 평균 → 블러	사진을 스크래치 유리 너머로 봄
중심 큼·주변 음수	중심 강조 → 샤프닝	윤곽선에 검은 펜 덧그림
좌-우 또는 상-하 부호 반대	변화량 → 엣지	"달라지는 곳만" 검출하는 차이 탐지기

2-5. Gaussian 필터

한마디로 — "중앙은 세게, 바깥은 살살" 가중 평균하는 블러

- 단순 평균 블러는 주변 9칸을 똑같이 섞어 티가 난다.
- 가우시안은 가운데 픽셀에 가장 큰 가중치, 멀어질수록 가중치가 줄어드는 자연스러운 블러.
- 비유: 종 모양 곡선(정규분포) — 산봉우리 모양의 가중치
- σ 가 크면 더 넓게·더 부드럽게 블러 (흐림 강도 손잡이)

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

수식을 말로 풀면

- $x^2 + y^2$: 중심에서의 거리 제곱 → 멀수록 커짐
- $\exp(-\text{거리}^2)$: 멀수록 지수적으로 작아짐 → 가중치 감소
- 앞의 $\frac{1}{2\pi\sigma^2}$: 전체 합을 1로 맞추는 정규화 상수

- 중심 가중, 주변 감소 — 평균 필터보다 자연스럽게 부드러움
- 잡음 제거 효과 탁월 → Sobel/Canny 전처리 표준
- `cv2.GaussianBlur`, `cv2.getGaussianKernel`

2-6. Sobel vs Canny 엣지 검출

Sobel과 Canny 엣지 검출: 차이 한 그림에 정리

① Sobel 마스크 (3×3)

Gx (가로 엣지)			Gy (세로 엣지)		
-1	0	+1	-1	-2	-1
-2	0	+2	0	0	0
-1	0	+1	+1	+2	+1

직관: Gx는 "왼쪽-오른쪽 밝기 차", Gy는 "위-아래 밝기 차"

크기: $||\nabla|| = \sqrt{I_x^2 + I_y^2}$ 방향: $\theta = \text{atan2}(I_y, I_x)$

약점: 노이즈에 예민 → 블러 없이 쓰면 잡음도 엣지로.

② Canny 파이프라인 (Sobel을 감싸는 전처리+후처리)



각 단계가 해결하는 문제

- 1단계: 노이즈 한 점도 엣지로 오인되는 것 방지
- 2단계: 경사도 자체를 계산 (Sobel과 같다)
- 3단계: 한 엣지가 두꺼운 띠로 나오는 현상 제거
- 4단계: 약한 엣지 중에도 강한 엣지와 연결된 것만 살림

핵심 비교

- Sobel: 그래디언트 크기 = 엣지 후보. 단순·빠름·잡음 취약.
- Canny: Sobel + 블러 + NMS + 이중임계. 얇고 깨끗한 엣지.

실습 패턴: `cv2.GaussianBlur` → `cv2.Canny` (블러 먼저!)

한마디로 — "밝기가 확 바뀌는 곳을 찾아 선으로 그린다"

- 엣지(edge)는 물체의 윤곽선. 밝기가 가장 가파르게 변하는 지점.
- 수학적으로는 기울기(**gradient**)가 큰 곳. 기울기는 "왼쪽/오른쪽 밝기 차"(가로 방향 I_x)와 "위/아래 밝기 차"(세로 방향 I_y).
- Sobel = 기울기 자체만 구하는 기초 도구
- Canny = 잡음 지우고 얇게 다듬어 연결까지 해주는 완성품

구분	Sobel	Canny
원리	기울기 (I_x, I_y) 직접 계산	Gaussian + gradient + NMS + hysteresis
출력	기울기 크기/방향	얇은 이진 엣지
완성도	기초 연산	완성된 파이프라인

Canny 4단계 — 반드시 순서대로 외우기

#	단계	왜 필요?	일상 비유
1	Gaussian blur	잡음은 미세한 밝기 튀김 → 블러로 먼저 지우지 않으면 가짜 엣지 폭발	사진 찍기 전 렌즈 청소

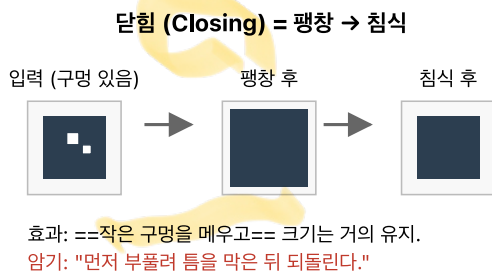
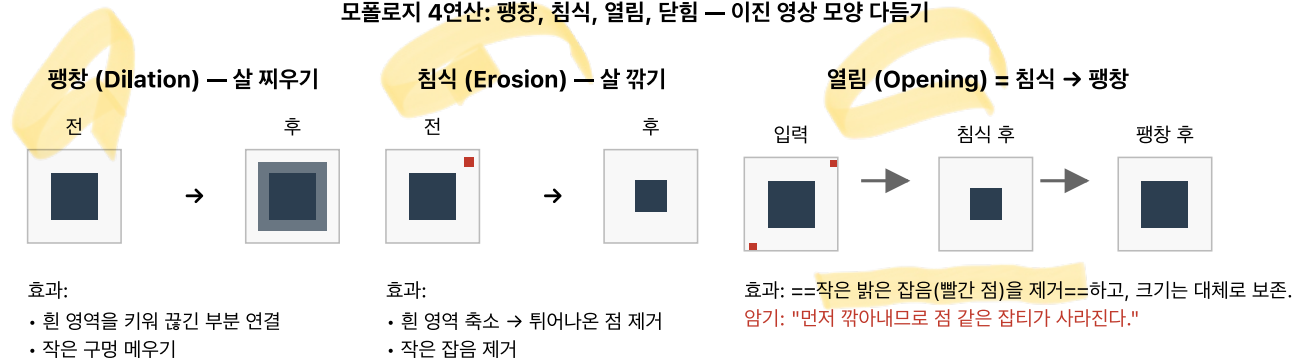
#	단계	왜 필요?	일상 비유
2	Gradient (Sobel)	어디서 얼마나 변하는가 숫자로 구하기	지도 등고선 가파른 곳 찾기
3	NMS (비최대 억제)	기울기 큰 띠가 두꺼움 → 가장 큰 값만 남기고 주변 지움	여러 겹 펜 자국에서 가장 진한 한 줄만 남김
4	Double threshold + Hysteresis	강한 엣지만 바로 채택, 약한 엣지는 강한 엣지와 이어져 있으면 채택	확실한 친구와 애매한 친구 — 둘이 아는 사이면 같이 초대

$$\|\nabla I\| = \sqrt{I_x^2 + I_y^2}$$

🔗 Sobel은 Harris의 I_x, I_y 로도 연결된다.

2-7. 모폴로지 4연산

모폴로지 4연산: 팽창, 침식, 열림, 닫힘 — 이진 영상 모양 다듬기



요약표

팽창(⊕):	흰 영역 키움 — 연결·메우기
침식(⊖):	흰 영역 깎음 — 잡음·돌기 제거
열림(⊖⊕):	침식→팽창 — ★작은 밝은 잡음 제거
닫힘(⊕⊖):	팽창→침식 — ★작은 어두운 구멍 메움

기억법:

열림은 "깎고 복구" → 튀어나온 것 제거
닫힘은 "부풀려 복구" → 움푹 들어간 것 제거

순서가 핵심: 반대로 하면 결과가 달라진다. 실습의 `cv2.morphologyEx(MORPH_OPEN/CLOSE)`로 한 번에 호출.

🔗 한마디로 — "이진(흑백) 이미지의 모양을 부풀리거나 깎아내는 도구"

- 주로 이진 이미지(물체=흰색, 배경=검정)에 쓴다.
- 팽창 = 흰 영역을 부풀림 / 침식 = 흰 영역을 깎음
- 비유: 포토샵의 "두껍게/얇게" 보정 또는 도장을 밀고 지우기

🔗 구조요소 (Structuring Element, B) — 모폴로지의 "도장 모양"

- 컨볼루션의 커널과 비슷한 역할 — 작은 이진 패턴(보통 3×3 또는 5×5)
- 예: 3×3 정사각형 구조요소 $B = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \rightarrow$ 8방향 이웃을 모두 반영
- 십자형 $\begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix} \rightarrow$ 4방향 이웃만 반영
- 팽창: B가 커버하는 영역에 하나라도 흰 픽셀이면 결과=흰색 (OR 연산)
- 침식: B가 커버하는 영역이 모두 흰색이어야 결과=흰색 (AND 연산)

- OpenCV: `cv2.getStructuringElement(cv2.MORPH_RECT, (3,3))`

연산	정의	효과	일상 비유
팽창 Dilation	밝은 영역 확장	굵긴 부분 연결, 틈 메움	글씨를 굵게 볼드
침식 Erosion	밝은 영역 축소	튀어나온 점 제거, 잡음 제거	글씨를 얇게 스키니
열림 Opening	$Erosion \rightarrow Dilation$	작은 밝은 잡음 제거	옷에 붙은 먼지 털어낸 뒤 복원
닫힘 Closing	$Dilation \rightarrow Erosion$	작은 어두운 구멍 메움	스웨터 작은 구멍 메운 뒤 다듬기

$$\text{Opening} = (A \ominus B) \oplus B \quad \text{Closing} = (A \oplus B) \ominus B$$

🔗 쉽게 외우는 법

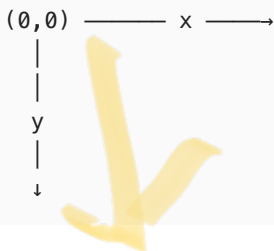
- 열림: 먼저 깎아서 작은 튀어나온 점을 떼어낸 뒤 원상 복구
- 닫힘: 먼저 부풀려서 작은 빈틈을 덮은 뒤 원상 복구

3. 기하 변환과 보간

3-0. 2D 이미지 좌표계 — 수학과 반대!

🔗 영상 좌표계의 특수성

- 수학 좌표계: $x \rightarrow$ 오른쪽, $y \rightarrow$ 위, 원점=왼쪽 아래
- 영상 좌표계: $x \rightarrow$ 오른쪽, $y \rightarrow$ 아래, 원점=왼쪽 위 (0,0)
- 픽셀 인덱싱은 (행 row, 열 col) = (y, x) 순 — `numpy img[y, x]`
- 이 때문에 회전 행렬의 sin 부호나 y 축 방향이 직관과 달라 보일 수 있음



3-1. 동차 좌표 (Homogeneous Coordinates)

🔗 한마디로 — "좌표 끝에 1 하나 덧붙여 이동까지 행렬로 표현하는 트릭"

- 2D 좌표 (x, y) 를 $(x, y, 1)$ 로 차원 1개 늘림
- 이렇게 하면 이동·회전·크기를 행렬 곱 하나로 모두 표현 가능
- 비유: 편지봉투 뒷면에 우표(=1)를 한 장 붙여야 우체통(행렬 곱)에 들어간다

$$(x, y) \rightarrow (x, y, 1), \quad (x, y, 1) \sim (kx, ky, k)$$

🔗 스케일 인자 k 의 의미

- $\sim$ 기호 = 스케일 무시 등가 — 0이 아닌 어떤 k 를 곱해도 같은 점으로 본다
- (kx, ky, k) 를 정규화하면 $(kx/k, ky/k) = (x, y) \rightarrow$ 원래 2D 좌표로 돌아옴
- $k = 1$ 이 가장 표준 형태 (마지막 성분을 1로 맞춤)

- 호모그래피의 핵심 성질: $\mathbf{x}' \sim H\mathbf{x}$ 에서 H 는 스케일 곱에 무관 $\rightarrow$ 자유도 8 (9-1)

- 왜 쓰는가: 이동·회전·크기를 모두 행렬 곱 하나로 통일
- 이동(translation)은 2D 선형변환으로 표현 불가 $\rightarrow$ 동차좌표 도입

3-2. 기하 변환 종류 — 3×3 행렬 정리

🔑 핵심 — 모든 2D 변환은 3×3 동차행렬로 통일. 동차좌표에 곱하면 한 번에 끝

변환	행렬 (3×3)	의미
이동 Translation $T(t_x, t_y)$	$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$	$(x,y) \rightarrow (x+t_x, y+t_y)$
회전 Rotation $R(\theta)$	$\begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$	원점 기준 θ 만큼 회전
크기 Scaling $S(s_x, s_y)$	$\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$	축 방향으로 배율

📌 복합 변환 — $p_1=(1,3)$ 을 이동(2,-1) 후 30° 회전

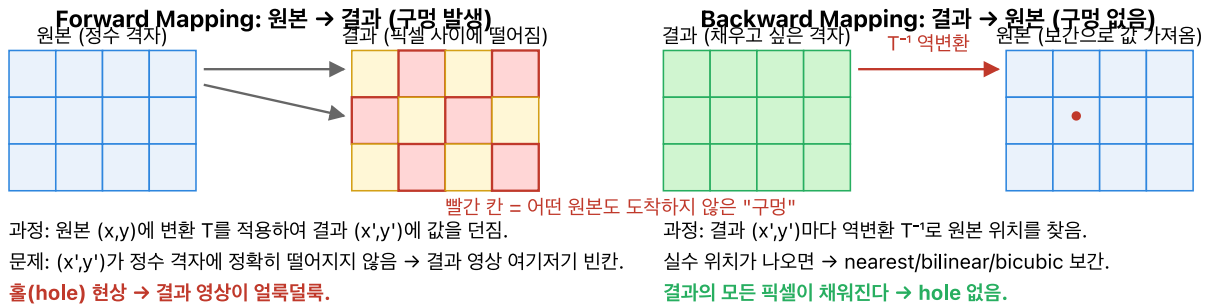
$$\begin{aligned}
 1. \text{ 이동 후: } T(2, -1) \cdot \begin{bmatrix} 1 \\ 3 \\ 1 \end{bmatrix} &= \begin{bmatrix} 3 \\ 2 \\ 1 \end{bmatrix} \\
 2. \text{ 회전 후: } R(30^\circ) \cdot \begin{bmatrix} 3 \\ 2 \\ 1 \end{bmatrix} &\approx \begin{bmatrix} 3.598 \\ 0.232 \\ 1 \end{bmatrix}
 \end{aligned}$$

- 합성 행렬 $A = R(30^\circ) \cdot T(2, -1)$ 를 미리 계산하면 한 번의 곱셈으로 처리
- 주의: 변환 순서가 바뀌면 결과도 바뀜 (행렬 곱은 비교환)

- OpenCV: `cv2.warpAffine(img, M, (w,h))`, `cv2.getRotationMatrix2D(center, angle, scale)`

3-3. Forward vs Backward Mapping

Forward vs Backward Mapping: 왜 backward가 더 안정적인가



시험 암기 포인트

- ==실전에서는 거의 backward mapping을 쓴다== (cv2.warpAffine, cv2.remap 등 내부도 같은 방식)
- forward는 구멍 → 후처리로 메워야 해서 비용이 더 크다.
- 한 문장: "결과 픽셀을 기준으로 원본을 뒤로 되찾아가면 모든 픽셀이 채워진다."

한마디로 — "결과 픽셀이 원본 어디서 왔는지 거꾸로 물어본다"

- **Forward:** 원본 픽셀을 하나씩 "너는 어디로 가?" 묻고 보낸다 → 새 이미지에 아무도 도착 안 한 빈 칸(구멍) 생김
- **Backward:** 결과 픽셀 하나하나가 "난 원본 어디에서 왔을까?" 역추적 → 빈 칸 없이 모두 채워짐
- 비유: 택배 돌릴 때, 보내는 쪽에서 "가라"고 밀면 누락 생김 vs 받는 쪽에서 "가져오겠다" 하면 누락 없음

방식	방향	문제
Forward	원본 → 결과 (보냄)	hole(구멍) 발생
Backward	결과 ← 원본 (가져옴)	구멍 없음, 보간 결합 자연스러움

실전에서는 backward mapping + 보간이 표준

3-4. 보간(Interpolation)

보간 방법 비교: Nearest vs Bilinear vs Bicubic

실수 좌표 (x', y') 에서 값을 가져올 때, 주변 픽셀을 몇 개 쓰느냐가 핵심.

① Nearest Neighbor (1개)

10	50	90
20	60	100

(x', y')

가까운 1개 픽셀 값 그대로 사용

결과 = 50 (가장 가까운 것)

- 장점: 매우 빠름
- 단점: 계단 현상, 각진 경계

픽셀 경계가 각짐 (blocky)

② Bilinear (4개)

10	50	90
20	60	100

주변 4개 픽셀의 가중평균

$a=dx, b=dy$

$$r = (1-a)(1-b) \cdot 10 + a(1-b) \cdot 50 + (1-a)b \cdot 20 + ab \cdot 60$$

- 부드럽지만 약간 흐림

경계가 부드러움

③ Bicubic (16개)

주변 $4 \times 4 = 16$ 개 샘플

3차 다항식 곡선으로 내삽

- 가장 부드럽고 자연스러움
- 가장 느리고 계산량↑
- 사진 확대에서 품질 최고

가장 매끄럽고 선명함

선택 기준

속도↑ Nearest — 균형 Bilinear (기본값) — 품질↑ Bicubic (사진/출판). OpenCV: cv2.INTER_NEAREST / LINEAR / CUBIC

한마디로 — "있는 픽셀 사이의 빈 값을 그럴듯하게 지어내기"

- 이미지를 확대하면 없는 위치의 밝기가 필요해진다 → 주변 픽셀로 추정
- 참조 픽셀을 많이 볼수록 부드럽지만 느려짐

방법	참조 픽셀	품질	속도	일상 비유
Nearest Neighbor	1개	계단 현상(blocky)	최고	가장 가까운 이웃 값을 그대로 베낌
Bilinear	4개	부드러움	보통	주변 4명의 평균 성적
Bicubic	16개	더 부드러움	느림	주변 16명 가중 평균 (곡선 맞춤)
Area	(축소 전용)	축소 시 안티앨리어싱	—	영역 면적만큼 골고루 섞기

- `cv2.INTER_NEAREST` / `INTER_LINEAR` / `INTER_CUBIC` / `INTER_AREA`

3-5. OpenCV 라이브러리 개요

OpenCV (Open Source Computer Vision Library)

- **2500+** 최적화 알고리즘 포함 — 영상 처리, 특징 추출, 객체 검출, 3D 재구성 등
- C++ 핵심 + Python/Java/JS 바인딩, CUDA GPU 가속 지원
- 오픈소스 (Apache 2.0) — 연구·상용 모두 무료 사용 가능
- 개발 연혁: Intel 1999 → Willow Garage → OpenCV.org

3-6. numpy 배열과 BGR 채널 — 시험 필수

OpenCV 이미지는 `numpy.ndarray`

- 흑백: `shape = (H, W)`, `dtype = uint8`
- 컬러: `shape = (H, W, 3)` — 마지막 축이 채널
- 채널 순서 = **BGR** (Blue-Green-Red), RGB 아님!
- 픽셀 접근: `img[y, x, c]` — 행(y)이 먼저, 열(x) 나중 (수학적 관행 반대)

BGR
RGB X

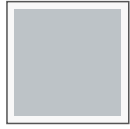
변환	OpenCV 코드
BGR → RGB (matplotlib용)	<code>cv2.cvtColor(img, cv2.COLOR_BGR2RGB)</code>
BGR → Gray	<code>cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)</code>
BGR → YUV	<code>cv2.cvtColor(img, cv2.COLOR_BGR2YUV)</code>

- 파일 읽기: `cv2.imread(path)` → 기본 BGR / `cv2.imread(path, 0)` → 흑백
- 표시: `cv2.imshow(name, img)` / `cv2.waitKey(0)`

4. Harris 코너 검출

Harris 코너 검출: 어떤 픽셀이 "코너"인가

세 종류의 영역



flat (평탄)
 $\lambda_1 \approx \lambda_2 \approx 0$



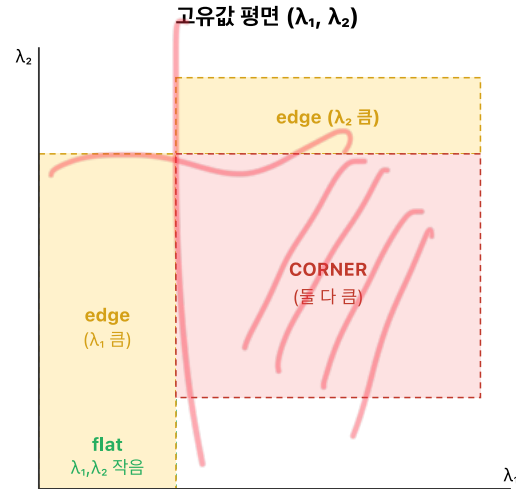
edge (에지)
 $\lambda_1 \gg \lambda_2 \approx 0$



corner (코너)
 λ_1, λ_2 둘 다 큼

작은 창을 조금씩 옮겨볼 때

- 평탄: 어느 방향으로 변화 없음
- 에지: 에지 방향은 변화 없고, 수직 방향만 변화
- 코너: 모든 방향으로 변화 → 두 고유값 모두 큼



Harris 응답함수 R (고유값 직접 계산 없이 사용)

$M = \sum_w [[Ix^2 \ IxIy] ; [IxIy \ Iy^2]] \leftarrow$ 구조 텐서 (structure tensor)

$R = \det(M) - k \cdot \text{trace}(M)^2$ ($k \approx 0.04 \sim 0.06$)

$\det(M) = \lambda_1 \lambda_2, \text{trace}(M) = \lambda_1 + \lambda_2$

• R 큰 양수 → 코너 | R 큰 음수 → 에지 | $R \approx 0$ → 평탄

암기: "고유값 두 개가 모두 커야 코너" — R은 그것을 하나의 점수로 압축.

한마디로 — "작은 창문을 이리저리 밀어보고, 어느 방향으로 밀어도 다르게 보이는 곳 = 코너"

- 점 주변에 돋보기(작은 창)를 올려놓고 좌/우/위/아래로 살짝 움직여본다
- 평평한 벽: 어디로 밀어도 그대로 → 창문 속 그림 변화 없음
- 선(에지) 위: 선을 따라 밀면 그대로, 수직으로 밀면 변화 → 한 방향만 변화
- 모서리: 어디로 밀어도 그림이 달라짐 → 여기가 코너

4-1. 직관

작은 창(window)을 점 주변에 놓고 살짝 움직였을 때:

영역	움직였을 때 변화	판정
평평 Flat	어느 방향이든 무변화	둘 다 작음
에지 Edge	한 방향만 큰 변화	하나만 큼
코너 Corner	모든 방향 큰 변화	둘 다 큼

4-2. 핵심 수학

$$E(u, v) \approx \sum_{x, y} w(x, y) [I(x + u, y + v) - I(x, y)]^2$$

수식 풀이 — "창문을 (u, v)만큼 밀었을 때 얼마나 달라졌나"

- $I(x + u, y + v) - I(x, y)$: 민 뒤 픽셀 - 원래 픽셀 차이
- $[\cdot]^2$: 부호 무시하고 절대적 차이
- $\sum w(x, y)$: 창문 안의 모든 픽셀에서 가중 합산
- 즉 E는 "이 방향으로 민 결과 얼마나 달라졌나" 총점

전개하면 구조 텐서 M:

$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

구조 텐서 M 의 의미

- I_x, I_y : 가로/세로 기울기 (Sobel로 계산)
- M 은 "이 창 주변이 어느 방향으로 얼마나 변하는가"를 요약한 2×2 행렬
- 고유값 λ_1, λ_2 = 변화의 세기 두 방향

Harris response:

$$R = \det(M) - k(\text{trace}(M))^2$$

R 공식 풀이

- $\det M = \lambda_1 \lambda_2$ (둘 다 클수록 큼) → 코너일수록 큼
- $\text{tr } M = \lambda_1 + \lambda_2$ (하나만 커도 큼) → 에지에도 큼
- 둘을 빼면 "둘 다 큰 경우만" 양수로 살아남음
- $k \approx 0.04$ 는 에지를 얼마나 걸러낼지 정하는 민감도

M 의 두 고유값 λ_1, λ_2 해석:

- λ_1, λ_2 둘 다 작음 → **flat**
- 하나만 큼 → **edge**
- 둘 다 큼 → **corner**

4-3. R 부호로 판정하기 — 시험 단골

R 값	λ_1, λ_2	판정
$R \gg 0$	둘 다 크고 비슷	Corner (코너)
$R < 0$	한쪽만 큼 ($\lambda_1 \gg \lambda_2$)	Edge (에지)
\$	R	\approx 0\$



$E(u,v) \rightarrow M \rightarrow R$ 파이프라인 흐름

1. $E(u,v)$: 창을 (u,v) 만큼 민 후의 변화량 합
2. 테일러 전개로 $E \approx [u \ v] M \begin{bmatrix} u \\ v \end{bmatrix}$ 로 근사 → M 등장
3. 고유값 분석: M 의 λ_1, λ_2 가 창 주변의 지역적 구조를 요약
4. R 계산: 고유값 계산 회피 → $\det, \text{tr}$ 로 빠르게 판정

4-4. Harris 파라미터 3종

파라미터	의미	기본값
blockSize	창(window) 크기 — M 합산 이웃 범위	2
ksize	Sobel 커널 크기 — I_x, I_y 계산 정밀도	3
k	에지 억제 상수	0.04 (0.04~0.06)

- 임계값(threshold): $R > \text{threshold} * R.\text{max}()$ 로 최종 코너 선택 — 보통 0.01~0.1

```
cv2.cornerHarris(gray, blockSize=2, ksize=3, k=0.04)
```

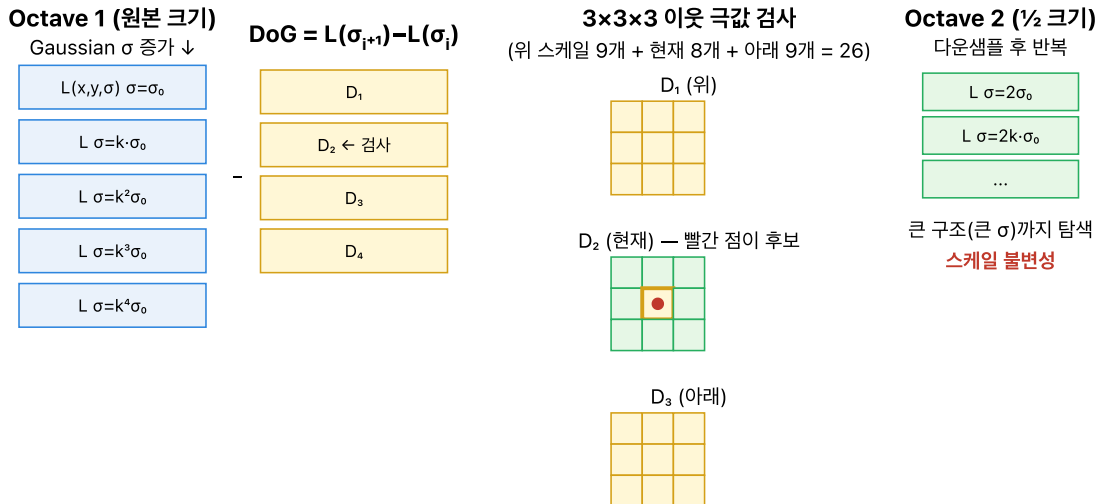
I_x, I_y 는 Sobel 기반

5. SIFT · ORB · 특징점 매칭

5-1. SIFT와 스케일 공간

SIFT의 Scale Space: Gaussian 피라미드와 DoG 극값

핵심: 서로 다른 σ 로 블러한 영상을 빼면 DoG (Difference of Gaussian)가 된다 → $3 \times 3 \times 3$ 이웃 26개와 비교해 극값이 특징점.



SIFT 4단계 한눈에

① Scale-space 극값 (DoG) → ② 부정확한/에지 후보 제거 → ③ 주방향 할당 → ④ 128차원 기술자(descriptor)

• 불변성: 스케일, 회전, 부분적 조명/시점

시험: "왜 DoG인가?" → LoG 근사, 계산 빠름 + 스케일 극값이 특징 영역에 대응.

한마디로 — "크기가 달라져도 같은 점을 찾아내는 특징점 검출기"

- 사진을 확대하거나 축소해도 같은 모서리가 검출돼야 한다 → 여러 흐림 정도($=\sigma$)로 여러 장 만들어 놓고 각 장마다 특징을 찾음
- 비유: 지도를 여러 축척(1:1000, 1:10000...)으로 그려 놓고, 축척이 달라도 표시되는 랜드마크만 고름
- DoG(Difference of Gaussian) = 두 흐림 강도의 빼기 → 그 scale에서 튀어나온 점이 드러남

- **SIFT (Scale-Invariant Feature Transform)**
- Gaussian을 여러 σ 로 반복 적용 → **scale-space** 구성
- 인접 레벨끼리 빼서 **DoG(Difference of Gaussian)** 생성
- 스케일이 달라도 같은 점을 안정적으로 검출

5-2. SIFT vs ORB 비교

SIFT vs ORB: 언제 무엇을 쓰나

항목	SIFT (2004)	ORB (2011)
검출기 (Keypoint)	DoG 극값 (Scale-space)	oFAST (FAST + 방향)
기술자 (Descriptor)	128차원 float (히스토그램 기반)	256bit BRIEF (이진 비교)
매칭 거리	L2 (유클리디안)	Hamming (XOR+popcount) — 매우 빠름
속도	느림 (수백 ms)	매우 빠름 (수 ms) — 실시간 가능
기술자 크기	128 × 4 = 512 byte	32 byte (1/16)
불변성	스케일+회전+조명 (강함)	회전 O, 스케일 ≈OK (피라미드)
라이선스	특허 만료됨 (2020~)	특허 없음 (OpenCV 기본)

한 줄 선택 기준

SIFT = 정확도 우선(SLAM 오프라인, 파노라마 고품질) · ORB = 속도 우선(모바일, 실시간 AR, VSLAM)

한마디로 — "디스크립터는 특징점의 지문. 지문 표기 방식이 다르다"

- 디스크립터: 특징점 주변 픽셀 패턴을 숫자 벡터 하나로 요약한 것 (그 점의 신분증)
- SIFT = 128개의 실수로 적은 지문 → 비교는 L2(유클리드) 거리
- ORB = 256비트(0/1) 비트열로 적은 지문 → 비교는 Hamming 거리(다른 비트 수)
- 비유: 긴 설명문 vs 짧은 바코드 — 바코드가 훨씬 빠르게 일치 여부 비교됨

항목	SIFT	ORB
디스크립터 타입	실수 벡터 (128D)	이진 비트열 (256 bits)
거리 척도	L2 거리	Hamming 거리
계산량	무거움	가벼움
회전 불변	○	○
스케일 불변	◎	○
실시간 처리	△	◎
특허	(과거 특허, 현재 자유)	자유

시험 포인트

- SIFT descriptor는 실수 → L2 거리 사용
- ORB descriptor는 이진 → Hamming 거리 사용 (XOR 후 1의 개수)

5-3. 매칭기(Matcher)

- BFMatcher — 전수 비교, 정확하지만 느림
- FLANN — 근사 최근접 이웃, 빠름
- cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=False) (ORB 용)

실습 — Feature_Matching_Benchmark.ipynb, ORB_Video_Tracking.ipynb

SIFT: cv2.SIFT_create() · ORB: cv2.ORB_create(nfeatures=2000)

5-4. Similarity Search — Faiss (Facebook AI Similarity Search)

🔗 한마디로 — "수억~수십억 개 고차원 벡터에서 가장 비슷한 것을 초고속으로 찾는 라이브러리"

- 대규모 특징점/임베딩 벡터 검색 & 클러스터링용 오픈소스
- BF/FLANN으로 감당 안 되는 스케일(수천만~수십억 벡터)을 근사 최근접 이웃(ANN) 알고리즘으로 처리
- GPU 가속(CUDA) + Quantization 압축(메모리 1/10 이하)

주요 특징

- ANN(Approximate Nearest Neighbor) 탐색을 위한 다양한 알고리즘 지원
- GPU 가속: CUDA를 활용해 수십억 개 연산을 병렬 처리
- 압축(Quantization): 벡터 용량을 1/10 이하로 줄여 메모리 효율 극대화

적용 분야

- 대규모 이미지 검색 · 실시간 대량 특징점 처리
- 추천 시스템 (예: 수천만 사용자 취향 벡터 중 나와 비슷한 유저 찾기)
- 영상/문장을 벡터로 변환해 내용 기반 검색

5-4-1. Faiss의 주요 인덱스(알고리즘) — 시험 필수 암기

인덱스	특징	추천 상황
IndexFlatL2	전수 조사(Exact Search). 가장 정확	데이터 적고(100만↓) 정확도 중요
IndexIVFFlat	데이터를 여러 구역(Cell) 으로 나눠 검색 범위 제한	속도-정확도 균형 필요
IndexPQ	Product Quantization. 데이터 압축으로 메모리 절약	대규모 데이터를 메모리에 올려야 할 때
IndexHNSW	그래프 기반 탐색. 현존 ANN 중 가장 빠르고 정확	메모리 여유 있고 초고속 검색 필요

🔗 인덱스 선택 직관

- 규모 작음 → Flat (brute force)
- 중간 규모 → IVF (구역 나누기)
- 초대형 + 메모리 제약 → PQ (압축)
- 초대형 + 메모리 여유 → HNSW (그래프)

5-4-2. Quantization(양자화) 세 종류

양자화 = 연속적/고차원 값을 대표값으로 근사하는 것 (용량 줄이기 위한 압축 기법)

(1) Scalar Quantization(스칼라 양자화)

- 값 하나하나를 낮은 비트수로 표현 (예: float32 → 3 bits / 8 bits)
- 가장 단순, 정보 손실 큼

(2) Vector Quantization(벡터 양자화) — Bag-of-Words의 핵심

1. 코드북(Codebook) 생성: 데이터 공간을 여러 구역으로 나누고 각 구역의 중심점(Centroid) 을 정함. 중심점들의 모음 = 코드북
 2. 인코딩(Encoding): 입력 벡터가 들어오면 코드북에서 가장 가까운 중심점을 찾음
 3. 치환: 원래 복잡한 벡터 대신 중심점의 인덱스만 저장/전송
- 영상 처리 예: k=4, 16, 64, 256 로 증가할수록 원본에 가까워짐 (Bag-of-Words 원리)

(3) Product Quantization(곱 양자화) — Faiss의 핵심 시험 단골

- 아주 긴 벡터를 여러 개 짧은 조각으로 나눈 뒤, 각 조각을 미리 정해둔 코드북으로 대체
- 원래 데이터를 수십 배 이상 줄일 수 있음

❧ 예시로 감 잡기

128차원 SIFT 특징점 10억 개 원본 크기 = $128 \times 4\text{byte} \times 10\text{억} \approx 512\text{ GB}$
 → PQ 적용하면 수십 GB 수준으로 압축 가능

PQ 과정 — 3단계

1. 하나의 긴 벡터를 동일한 크기의 작은 조각(Sub-vector) m개 로 나눔
2. 학습 데이터를 이용해 각 조각들의 Centroid들을 **K-means** 클러스터링으로 찾아냄
3. 실제 데이터를 저장할 때 각 조각을 가장 닮은 **centroid**의 인덱스로 바꿈

예: 50,000개 특징점, 각 1024차원 → 8개 sub-vector로 분할

PQ 거리 계산 — 룩업 테이블 방식

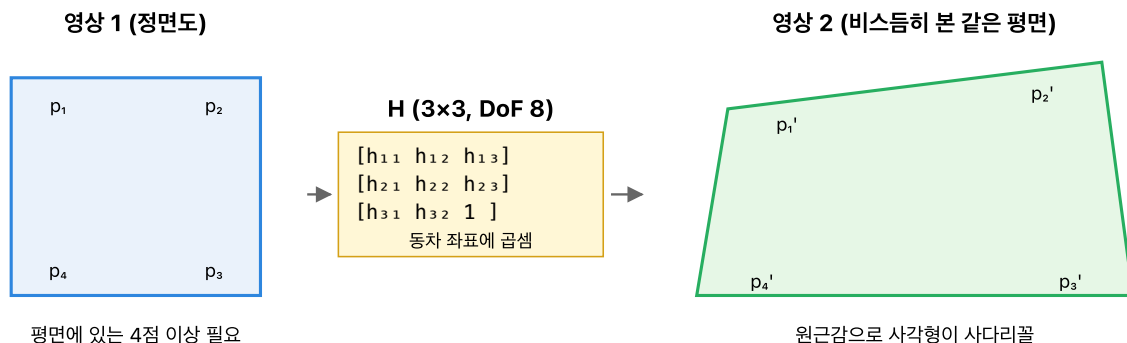
- Query vector의 각 조각과 DB의 각 조각 내 모든 centroid 간 **L2** 거리를 미리 계산해 테이블 구축
- 예: Query 조각 8개 × DB centroid 256개 → 테이블 크기 **256 × 8 = 2048**
- DB 데이터와 비교 시: 각 조각의 인덱스에 해당하는 거리값을 테이블에서 가져와 모두 더함
- 예: DB 데이터 하나가 [12, 105, 3, 210, 45, 90, 17, 200] 이라면
 1. 테이블의 첫 번째 조각에서 **12번째** 인덱스 거리값을 가져옴
 2. 이를 여덟 번째 조각까지 반복해 모두 더함

🔗 시험 예상 포인트

- **Faiss** = 대규모 ANN 라이브러리, **Facebook AI** 개발
- **4대 인덱스**: Flat(정확) · IVF(구역) · PQ(압축) · HNSW(그래프)
- **PQ 3단계**: 조각 분할 → K-means centroid → centroid 인덱스로 치환
- **PQ 장점**: 메모리 수십 배 절감, 룩업 테이블로 거리 계산 고속화
- **Vector Quantization** 과 **Product Quantization** 차이 구분

6. Homography

호모그래피 H: 평면과 평면 사이의 변환 (3×3 행렬)



핵심 식과 자유도

$$[x' \ y' \ w'] = H \cdot [x \ y \ 1]^T \rightarrow \text{실제 좌표는 } x'/w', y'/w'$$

- 자유도(DoF) = 8: 마지막 $h_{33}=1$ 로 정규화 → $9-1=8$
- 대응점 1쌍 → 2식 → 4쌍이면 8식 → 정확히 풀 수 있음 (4점법)
- 5쌍 이상이면 과결정 → 최소제곱(Least Squares) 또는 RANSAC으로 풀기

활용: 파노라마 스티칭, 문서 보정(perspective warp), AR 평면 기반 합성, 위성영상 정합. OpenCV: `cv2.findHomography` + `cv2.warpPerspective`

🔗 한마디로 — "같은 평면을 다른 각도에서 찍었을 때 어떻게 찌그러져 보이는지 알려주는 3×3 표"

- 책상 위 책 표지를 정면에서 찍은 사진과, 비스듬히 찍은 사진이 있다고 하자.
- 표지의 꼭짓점 4개가 어디로 이동했는지만 알면 표지 위 모든 점의 이동을 계산할 수 있다 — 이 계산 규칙이 호모그래피 H
- 활용: 파노라마 합성, AR 평면 인식, 문서 스캔 보정(perspective correction)

6-1. 정의

같은 평면 위 물체가 서로 다른 시점의 두 영상으로 찍혔을 때의 대응 관계:

$$\mathbf{x}' \sim H\mathbf{x}, \quad H \in \mathbb{R}^{3 \times 3}$$

- $\sim$ 는 스케일 무시 (동차 좌표)
- H 는 9개 원소 - 1 자유도(스케일) = 자유도 8

6-2. 왜 최소 대응점 4쌍인가

점 1개 = 독립 식 2개 제공

$$\text{자유도 } 8 \div \text{점당 식 } 2 = \boxed{4 \text{ 쌍}}$$

🔗 비전공자용 직관

- 풀어야 할 미지수 8개. 미지수 1개마다 식이 1개 필요 → 식 8개 필요
- 점 하나당 (x', y') 두 좌표가 대응되므로 식 2개를 준다
- $8 \div 2 = 4$ 쌍이면 딱 방정식이 완성 — 그래서 "직사각형 4꼭짓점만 찍으면" 호모그래피 결정

🔗 암기

- **Homography** = 평면 물체의 영상 간 변환
- 최소 대응점 4쌍
- 3쌍이면 아핀(Affine) 변환만 결정됨

6-3. H 풀이 — 선형 방정식 $A\mathbf{h} = 0$

🔗 핵심 — $H\mathbf{a}=\mathbf{b}$ 관계를 $\mathbf{b} \times H\mathbf{a} = 0$ 로 재구성해 선형 시스템 만들

- $\mathbf{x}' \sim H\mathbf{x}$ 에서 $\sim$ (스케일 무시) 때문에 곧바로 $H\mathbf{x} = \mathbf{x}'$ 로 풀 수 없음
- 외적(cross product) 성질 이용: 두 동차 벡터가 평행이면 외적 = 0
- 즉 $\mathbf{x}' \times (H\mathbf{x}) = 0 \rightarrow$ 각 대응점마다 선형 식 2개 유도

점 하나 $(x, y) \leftrightarrow (x', y')$ 에서 얻는 두 식:

$$\begin{aligned} -x h_{11} - y h_{12} - h_{13} + x' x h_{31} + x' y h_{32} + x' h_{33} &= 0 \\ -x h_{21} - y h_{22} - h_{23} + y' x h_{31} + y' y h_{32} + y' h_{33} &= 0 \end{aligned}$$

→ 4쌍이면 8×9 행렬 A , 미지수 9개($\mathbf{h}$)인 동차 시스템 $A\mathbf{h} = 0$

🔗 SVD로 $\mathbf{h}$ 구하기 — $h_{33} = 1$ 스케일 고정

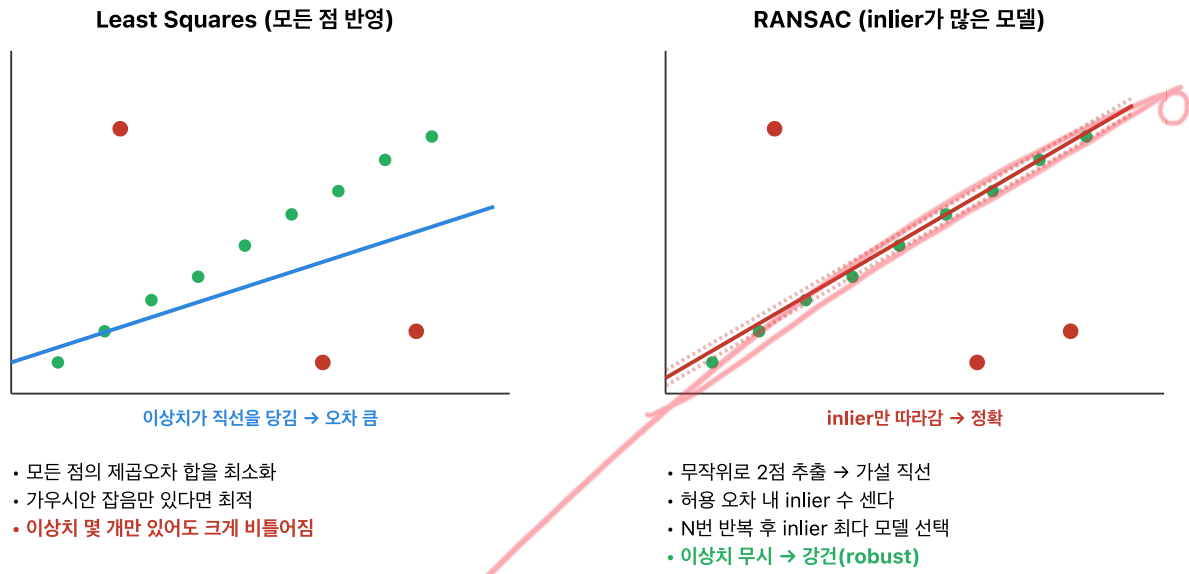
- $A = U\Sigma V^T$ 로 SVD 분해
- 최소 특이값에 대응하는 V 의 마지막 열이 해 $\mathbf{h}$
- 그 후 h_{33} 으로 나눠 스케일 정규화 → H 완성

- 이유: $\|A\mathbf{h}\|$ 를 최소화하는 단위벡터는 가장 작은 특이값 방향

7. Least Squares vs RANSAC

Least Squares vs RANSAC: 이상치가 있을 때 무엇이 살아남는가

대응점 중에는 매칭이 잘못된 이상치(outlier)가 섞인다. LS는 모두 반영 → 직선이 크게 비틀어짐. RANSAC은 무작위로 뽑아 가장 많은 inlier를 얻는 모델 선택.



언제 무엇을 쓰나

- 깨끗한 데이터 + 가우시안 잡음 → LS (빠름)
- 매칭에 틀린 쌍이 섞이는 호모그래피/F행렬 추정 → RANSAC 필수 (OpenCV: `cv2.findHomography(..., cv2.RANSAC)`)

한마디로 — "자(직선) 하나로 점들을 맞추는 두 가지 철학"

- Least Squares = "모든 점을 믿고" 오차 제곱 합이 최소인 직선 → 이상치 한두 개가 직선을 확 잡아당김
- RANSAC = "다수결 투표" → 임의로 뽑아 가설 세우고, 그 가설을 가장 많이 지지하는(inlier) 가설 채택
- 비유: 회식 메뉴 정하기 — 평균 취향(LS) vs 최다득표 메뉴(RANSAC)

7-1. Least Squares

$$\min_{\theta} \sum_i \|y_i - f(x_i; \theta)\|^2$$

선형 모델 $A\mathbf{x} = \mathbf{b}$ 의 폐형식 해(Closed-form)

$$\mathbf{x}^* = (A^T A)^{-1} A^T \mathbf{b}$$

- 유도: 오차 $\|A\mathbf{x} - \mathbf{b}\|^2$ 를 $\mathbf{x}$ 로 미분 → $2A^T(A\mathbf{x} - \mathbf{b}) = 0$
- $A^T A$ 를 정규방정식(normal equation)의 행렬이라 부름

- 장점: 수식 깔끔, 계산 효율 (한 번의 행렬 연산)
- 단점: outlier에 매우 약함
 - 큰 오차가 제공되면서 영향이 폭증

7-2. RANSAC (RANdom SAMple Consensus) — 4단계 방법론

"전체를 다 믿지 말고, 일부로 가설 세워보고, 지지(inlier)가 가장 많은 모델을 고르자"

#	단계	내용	예 (Homography)
1	Hypothesis (가설)	최소 샘플 랜덤 선택	대응쌍 4개 뽑기
2	Model (모델)	샘플로 모델 생성	H 계산 (SVD)
3	Verification (검증)	inlier 개수 측정: $\ b - Ha\ < t$	임계 t 이하 점 세기
4	Best Selection (선택)	반복 후 최다 inlier 모델 채택	최적 H^*

반복 횟수 결정: $N = \frac{\log(1-p)}{\log(1-(1-\epsilon)^s)}$ (p =성공확률, ϵ =outlier비율, s =최소샘플수)

🔄 RANSAC이 강건한 이유

- outlier는 확률적으로 잘 뽑히지 않음 — 4개 모두 inlier인 가설이 한 번이라도 나오면 승리
- 제곱 오차가 아니라 이진 투표(inlier/outlier)로 평가 → 이상치 영향 차단
- 최종 단계에 inlier만으로 LS 재추정 → 정확도↑

```
# RANSAC + Homography 실전 파이프라인
H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
inliers = mask.ravel() == 1
```

방법	장점	단점
Least Squares	깔끔·빠름	outlier에 약함
RANSAC	outlier에 강함	반복 계산, 파라미터 민감

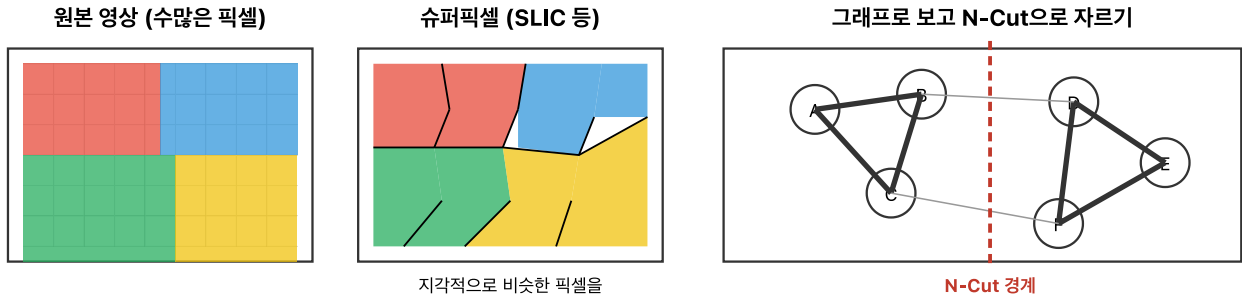
🔧 실습

```
cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 3.0)
```

매칭점이 많아도 outlier를 제거하지 않으면 Homography가 망가진다.

8. Superpixel과 N-Cut

슈퍼픽셀 + Normalized Cut: 픽셀을 덩어리로 묶어 분할



슈퍼픽셀 (Superpixel)

- 수백만 픽셀을 수백 개의 덩어리로 축약
- 각 덩어리 안의 픽셀은 색·위치가 비슷함
- 이후 처리(분할, 추적 등)의 기본 단위
- 대표 알고리즘: SLIC (k-means 기반), Felzenszwalb

핵심 이득: 계산량 ↓, 경계 보존 ↑

Normalized Cut

- 영상을 그래프로: 노드=슈퍼픽셀, 에지=유사도
- 단순 Min-Cut: 작은 그룹을 선호해 치우침
- N-Cut: $\text{cut}(A, B) / \text{vol}(A) + \text{cut}(A, B) / \text{vol}(B)$
- "크기로 정규화" → 균형 잡힌 분할

내부 유사도 ↑, 경계 유사도 ↓인 분할 찾기

한눈에 보는 파이프라인

픽셀 → ① SLIC으로 슈퍼픽셀 생성 → ② 슈퍼픽셀 간 유사도 그래프 → ③ N-Cut으로 균형 분할 → 객체 영역

요점: 슈퍼픽셀은 "데이터 축약기", N-Cut은 "균형 잡힌 분할 기준".

8-1. Superpixel (SLIC) — 파라미터 이해

- 픽셀보다 크고, 객체보다 작은 지역 단위
- SLIC(Simple Linear Iterative Clustering)
- 각 픽셀을 (R, G, B, x, y) 5차원 특징으로 보고 k-means 군집화
- 장점: 계산량 감소, 구조 보존, 후속 분할에 유리

파라미터	의미	영향
K (n_segments)	생성할 superpixel 개수	클수록 조각이 작고 세밀
Compactness (m)	색/위치 가중치 비율	클수록 공간 균일(정방형), 작을수록 색 경계 우선
sigma	사전 Gaussian smoothing	노이즈 억제

- 거리 척도: $D = \sqrt{d_c^2 + (d_s/S)^2 m^2}$ (d_c =색 거리, d_s =공간 거리, S =격자 간격)

8-2. Graph 표현 — 영상을 그래프로 (N-Cut의 전제)

영상 → 그래프 변환

- 정점(vertex): 각 픽셀 또는 superpixel
- 간선(edge): 이웃 정점을 연결
- 가중치 w_{ij} : 두 정점의 유사도 (색·밝기·공간 거리)
- 예: $w_{ij} = \exp(-\|F_i - F_j\|^2 / \sigma_F^2) \cdot \exp(-\|X_i - X_j\|^2 / \sigma_X^2)$

8-3. N-Cut (Normalized Cut) — 공식

$$\text{Ncut}(A, B) = \frac{\text{cut}(A, B)}{\text{assoc}(A, V)} + \frac{\text{cut}(A, B)}{\text{assoc}(B, V)}$$

- $\text{cut}(A, B) = \sum_{i \in A, j \in B} w_{ij}$ (두 그룹 사이 간선 가중치 합)
- $\text{assoc}(A, V) = \sum_{i \in A, j \in V} w_{ij}$ (A와 전체 사이 간선 합 — A의 크기)
- 단순 cut은 작은 조각(1개 정점만 떼어내기)에 쏠림 → 분모로 정규화해 균형 있는 분할 유도

한계	설명
계산 비용	고유값 분해 필요 — 큰 그래프에서 $O(N^3)$
파라미터 민감	σ_F, σ_X 튜닝 어려움

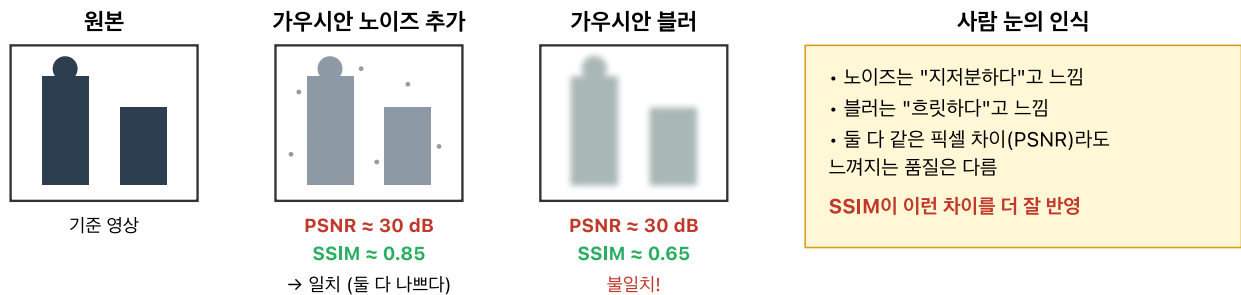
- `graph.cut_normalized(...)`

🔗 파이프라인

SLIC으로 잘게 조각내고 → Graph 구성 → N-Cut으로 의미 있게 합치거나 자름.

9. PSNR과 SSIM

PSNR vs SSIM: 왜 같은 PSNR도 화질이 달라 보이나



PSNR (Peak Signal-to-Noise Ratio)

$MSE = (1/N) \sum (I - K)^2$
 $PSNR = 10 \cdot \log_{10} (255^2 / MSE) \text{ [dB]}$

- 픽셀 차이만 본다 (전역 평균)
- 값이 클수록 좋음 (30dB↑ 무난)
- 구조/대비 변화 반영 못함

장점: 단순·빠름, 수학적으로 깔끔
단점: 사람 지각과 자주 어긋남

SSIM (Structural Similarity)

$SSIM = \text{luminance} \cdot \text{contrast} \cdot \text{structure}$
 $l(x, y) \cdot c(x, y) \cdot s(x, y)$

- 밝기 평균, 분산(대비), 공분산(구조)
- 지역 윈도우로 계산 후 평균
- -1 ~ 1, 1이면 완벽히 같음

장점: 블러·왜곡을 PSNR보다 잘 반영
단점: 계산이 다소 복잡

시험 암기: "PSNR은 픽셀 차이, SSIM은 구조 유사성." — 실전 논문에서는 둘 다 같이 쓰는 경우가 많음.

🔗 한마디로 — "복원 이미지를 원본과 비교해 점수 매기는 두 가지 방식"

- **PSNR** = "픽셀 값이 평균적으로 얼마나 어긋났나" (숫자 오차 채점관) — 값이 클수록 좋음. 30dB↑ 무난, 40dB↑ 거의 같음
- **SSIM** = "사람 눈이 느끼는 구조가 얼마나 비슷한가" — 값이 1에 가까울수록 좋음
- 왜 둘 다 필요? PSNR이 똑같아도 블러와 노이즈 왜곡의 느낌은 다름 → SSIM이 그 차이를 잡아낸다.

9-1. PSNR — 숫자 오차 기반

$$MSE = \frac{1}{MN} \sum_{x,y} (I(x,y) - K(x,y))^2$$

$$PSNR = 10 \log_{10} \left(\frac{MAX_I^2}{MSE} \right)$$

🔗 수식 풀이

- **MSE**: 두 이미지 픽셀 차이를 제곱해 평균 → "평균 오차 크기"
- MAX_I : 픽셀 최대값 (보통 255)
- $\log_{10}$ 을 곱한 이유: 오차 범위가 너무 넓어서(1~10000 배) **dB(데시벨)**로 압축해 읽기 쉽게 한 것

- MSE가 작아질수록 분수가 커지고 → **PSNR**이 커진다

- MSE ↓ → PSNR ↑
- **PSNR**이 클수록 원본에 더 가깝다

9-2. SSIM — 시각 구조 기반

$$\text{SSIM}(x, y) = [l(x, y)]^\alpha [c(x, y)]^\beta [s(x, y)]^\gamma$$

3요소를 함께 측정:

- **l** — 휘도 **Luminance**
- **c** — 대비 **Contrast**
- **s** — 구조 **Structure**

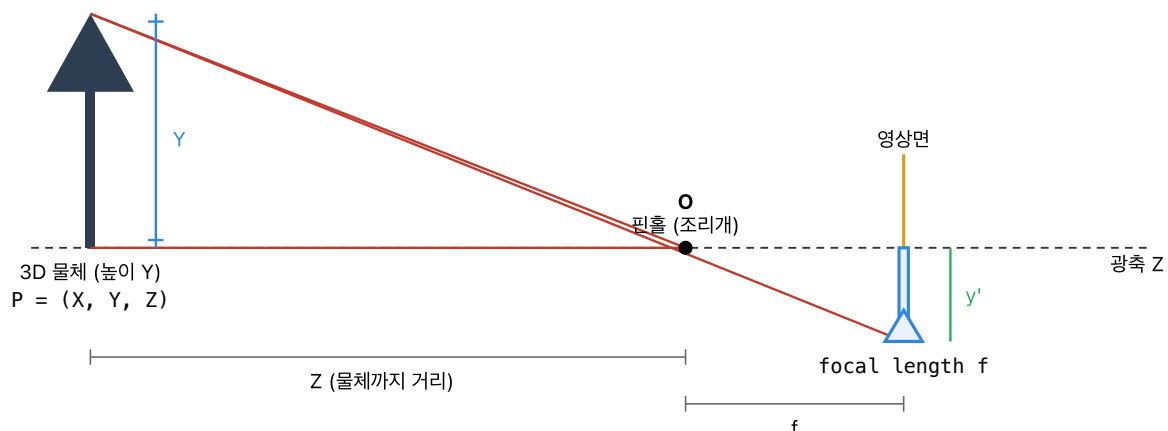
지표	관점	강점
PSNR	픽셀 오차(MSE)	수치 계산 단순
SSIM	사람 눈의 구조 인식	경계/구조 손상 민감

한 줄 요약

PSNR = 숫자 오차 중심 · **SSIM** = 사람 눈 기준 구조 중심

10. 핀홀 카메라 모델

핀홀 카메라 모델: 3D 세계 → 2D 영상의 투영



닮음삼각형의 결론 (투영 방정식)

$$x' = f \cdot X / Z, y' = f \cdot Y / Z \leftarrow Z \text{로 나눠서 "멀면 작게 보인다"}$$

핵심: Z로 나누는 이 한 번의 나눗셈이 원근감(perspective)을 만든다. 직각평행 → 사다리꼴 되는 이유.

한마디로 — "3D 점이 영상 위 어디에 찍히는가를 삼각비로 계산"

- 작은 구멍(핀홀) 하나로 3D 공간이 거꾸로 뒤집힌 **2D 상**으로 맺힌다 — 가장 단순한 카메라 모델
- 핵심: 멀리 있는 것은 작게, 가까이 있는 것은 크게 찍힌다 (비례 축소)
- 비유: 손전등으로 벽에 손 그림자 만들기 — 손을 벽에서 멀리 떼면 그림자가 작아짐

10-0. 카메라 구조 — 영상평면 vs 가상영상평면

구분	위치	상의 방향
영상평면 (Image plane)	핀홀 뒤	상하좌우 반전
가상영상평면 (Virtual image plane)	핀홀 앞	상하좌우 동일

왜 가상영상평면을 쓰는가?

기하학적으로 두 평면은 동일한 투영 결과를 가진다. 반전 없이 직관적으로 분석할 수 있어 수식 전개에 편리하다.

10-1. 투영식

$$x = \frac{fX}{Z} + p_x, \quad y = \frac{fY}{Z} + p_y$$

수식 풀이

- $\frac{X}{Z}$: 3D 점의 가로 위치를 깊이로 나눔 → "멀리 있을수록 작게 보이는 비율"
- $\times f$: 초점거리 곱해서 픽셀 단위로 변환
- $+p_x$: 영상 중심(주점)으로 원점 옮기기

- (X, Y, Z) : 3D 점 (카메라 좌표)
- (x, y) : 영상 평면의 픽셀 좌표
- f : 초점거리(focal length)
- (p_x, p_y) : 주점(principal point) — 주축이 영상과 만나는 점

10-2. 직관

- Z 가 크면 $(X/Z, Y/Z)$ 작아짐 → 멀리 있는 물체는 작게 찍힘
- 물체가 정면에서 멀어져도 비례로 축소

10-3. 깊이 모호성 (Depth Ambiguity)

(X, Y, Z) 를 같은 배율 α 로 키워도:

$$\frac{f(\alpha X)}{\alpha Z} = \frac{fX}{Z}$$

→ 영상에서는 동일해 보임

단일 카메라만으로는 절대 깊이 복원 불가

그래서 스테레오(양안) 또는 깊이 센서가 필요하다.

10-4. 카메라 캘리브레이션 — 내부/외부 파라미터

한마디로 — "카메라가 자신의 특성을 파악하는 작업"

- 체커보드 같은 알려진 패턴을 여러 각도로 촬영 → 파라미터 역산

내부(intrinsic) 파라미터 **K** — 카메라 고유 특성 (픽셀 단위)

$$K = \begin{bmatrix} f_x & s & p_x \\ 0 & f_y & p_y \\ 0 & 0 & 1 \end{bmatrix}$$

- f_x, f_y : x/y 방향 초점거리 (픽셀), s : skew (거의 0), (p_x, p_y) : 주점

외부(extrinsic) 파라미터 $[R|t]$ — 카메라의 세계 속 자세 (3×4)

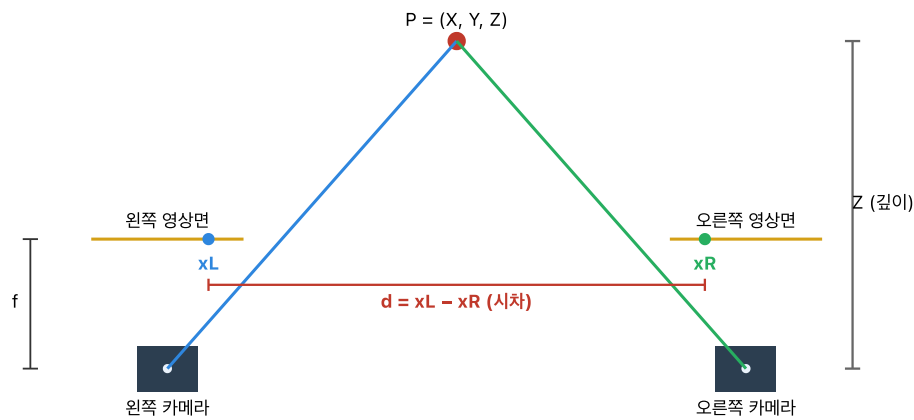
- R : 3×3 회전, t : 3×1 이동 → 월드 좌표를 카메라 좌표로 변환

전체 투영 행렬: $P = K[R|t]$ (3×4)

- OpenCV: `cv2.calibrateCamera(...)` → `ret, K, dist, rvecs, tvecs` 반환
- 왜곡(distortion)도 함께 추정: `radial( $k_1, k_2, k_3$ ) + tangential( $p_1, p_2$ )`

11. Stereo Vision — 깊이 추정

스테레오 비전: 시차(disparity)로 깊이(Z) 계산하기



스테레오 깊이 공식 (rectified 전제)

$$Z = f \cdot B / d$$

- f = 초점거리(공통), B = 두 카메라 간격, d = 같은 점이 좌/우 영상에서 나타나는 x 좌표 차이

핵심: 시차 d 가 클수록 물체가 가깝다($Z \downarrow$). $d=0$ 이면 $Z=\infty$ (무한히 먼 점).

- 전제: 두 영상이 rectified(에피폴라 선이 수평) → 수평 탐색만으로 대응점 찾을 (블록 매칭, SGBM 등).

🔄 한마디로 — "두 눈으로 본 시차(x 좌표 차이)만 있으면 깊이가 나온다"

- 왼쪽 눈·오른쪽 눈을 번갈아 감으면 손가락이 좌우로 움직여 보인다. 가까울수록 많이 움직이고 멀수록 덜 움직인다.
- 이 움직인 거리 = **disparity** → 거꾸로 뒤집어 깊이 Z 를 계산한다.
- 비유: 두 귀의 소리 도착 시간 차이로 음원 방향을 판단하는 것의 시각판

11-1. Disparity란

좌·우 카메라에서 같은 점이 찍힌 픽셀 위치의 차이.

- 가까운 물체: disparity ↑ (좌우에서 많이 어긋남)
- 먼 물체: disparity ↓ (덜 어긋남)

11-2. 깊이 공식 — 반드시 암기

가정: 두 카메라의 초점거리 동일, 평행 배치, rectification 완료

$$Z = \frac{fB}{d}$$

- Z : 깊이
- f : 초점거리
- B : baseline (두 카메라 중심 거리)
- d : disparity

🔄 수식을 직관으로

- 분자 fB 는 카메라 설정으로 고정된 상수
- 분모 d (시차)가 크면 $\rightarrow Z$ 가 작다(가까움) / d 가 작으면 $\rightarrow Z$ 가 크다(멀다)
- 깊이와 시차는 반비례 — 멀리 있는 별은 두 눈으로 봐도 거의 안 움직임($d \rightarrow 0$)

🔄 의미

d 가 크면 Z 는 작다 (가까움) · d 가 작으면 Z 는 크다 (멀다)
 $\rightarrow$ 깊이는 **disparity**에 반비례

11-3. 수치 예시 (슬라이드 단골)

$f = 1000$, $B = 65 \text{ mm}$, $d = 50$ 이면:

$$Z = \frac{1000 \times 65}{50} = 1300 \text{ mm}$$

🔗 왜 $B=65\text{mm}$? — 인간 눈의 평균 동공간 거리(IPD)

성인 평균 약 **6.5cm** — 생체 시차 시스템을 모사한 설정. 드론/로봇 스테레오 카메라도 이 값을 출발점으로 삼는다.

11-3-1. 깊이 공식 적용의 3가지 필수 가정

#	가정	위배 시
1	두 카메라의 초점거리 동일 ($f_L = f_R$)	좌우 스케일 불일치
2	두 카메라 광축이 평행 (베이스라인에 수직)	시차가 비수평으로 흩어짐
3	Rectification 완료 — 동일 y좌표에 대응점 존재	탐색 범위 폭발

11-3-2. Epipolar 제약 (Epipolar Constraint)

🔄 핵심 — "좌 영상의 한 점에 대응하는 우 영상의 대응점은 특정 직선(epipolar line) 위에만 존재한다"

- 탐색 공간을 **2D** $\rightarrow$ **1D**로 축소 $\rightarrow$ 매칭 속도/정확도 급증
- Rectified 스테레오에서는 이 직선이 수평선(같은 행)이 됨
- 수학: $\mathbf{x}_R^T \mathbf{F} \mathbf{x}_L = 0$ ($\mathbf{F}$ = Fundamental Matrix, 3×3)

11-4. Disparity Map

- 모든 픽셀에 disparity를 기록한 2D 맵
- 희소(sparse) 특징점 매칭만으로는 부족 $\rightarrow$ dense stereo 기법 필요

⚠️ 실전의 어려움

SIFT/ORB 같은 희소 매칭은 강력하지만 모든 픽셀 disparity를 주지 못함.
Dense disparity는 block matching, SGM 등 별도 기법이 필요.

11-5. 3D 좌표계 체인 변환

여러 좌표계를 거치는 변환은 누적 행렬 곱으로 표현

$$X_4 = R_4(R_3(R_2(R_1X + t_1) + t_2) + t_3) + t_4$$

- 월드 → 카메라 → 이미지 평면 → 픽셀 좌표로 차례로 변환
- 행렬 곱으로 미리 합성해두면 단일 연산으로 처리 가능
- 4×4 동차 변환 행렬 $\begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}$ 로 표현하면 깔끔히 연쇄

11-6. COLMAP — SfM + MVS 파이프라인

단계	이름	출력
1	SfM (Structure from Motion)	여러 사진의 카메라 자세 + sparse 3D 점군
2	MVS (Multi-View Stereo)	dense 3D 점군, 메시

- 파이프라인: 특징점 검출 → 매칭 → RANSAC + 카메라 자세 복원 → Bundle Adjustment → Dense reconstruction
- 3D Gaussian Splatting, NeRF의 전처리로 필수 (카메라 포즈 제공)

11-7. 컴퓨터비전 응용 분야 한눈 보기

객체 검출 (Object Detection) — 바운딩 박스 + 클래스

분류	대표 모델	특징
2-stage	R-CNN, Fast/Faster R-CNN	RPN으로 후보 생성 → 정교, 느림
1-stage	YOLO, SSD, RetinaNet	한 번에 검출 → 빠름, 실시간
Transformer	DETR	Set prediction, anchor-free

분할 (Segmentation)

종류	정의	예
Semantic	같은 클래스 = 같은 라벨	모든 사람 픽셀 → "person"
Instance	같은 클래스여도 개체별 구분	사람1, 사람2
Panoptic	Semantic + Instance 통합	모든 픽셀에 (클래스, 인스턴스)

Pose Estimation — 사람 관절 위치 (키포인트) 검출

3D 비전 최전선

기술	핵심
3D Gaussian Splatting	3D 가우시안 집합으로 장면 표현 — 실시간 렌더
ViT (Vision Transformer)	이미지를 패치 토큰으로 → Transformer 처리
SLAM	Simultaneous Localization And Mapping — 자율주행/AR/로봇

13. 비교형 핵심 카드

13-1. Sobel vs Canny

- **Sobel**: 기울기 계산만
- **Canny**: Gaussian + gradient + NMS + hysteresis (완성 파이프라인)

13-2. SIFT vs ORB

- **SIFT**: 정확·무거움 · L2
- **ORB**: 빠름·가벼움 · Hamming

13-3. PSNR vs SSIM

- **PSNR**: 숫자 오차
- **SSIM**: 시각 구조

13-4. Least Squares vs RANSAC

- **LS**: 모든 점 신뢰 → outlier에 약함
- **RANSAC**: inlier 중심 선정 → outlier에 강함

13-5. Opening vs Closing

- **Opening**: 작은 밝은 잡음 제거 (Erosion → Dilation)
- **Closing**: 작은 어두운 구멍 메움 (Dilation → Erosion)

13-6. Forward vs Backward Mapping

- **Forward**: 구멍 발생
- **Backward**: 구멍 없음, 실전 표준

14. 시험 직전 체크리스트

✓ 아래를 말로 설명할 수 있으면 준비 완료

- ☐ 감마 보정 수식 $+ \gamma < 1$ vs $\gamma > 1$ 효과
- ☐ YUV에서 Y만 처리하는 이유
- ☐ Gaussian 전처리가 필요한 이유
- ☐ Sobel vs Canny 차이 + Canny 4단계
- ☐ 모폴로지 4연산 정의 + opening/closing 합성
- ☐ 동차 좌표를 쓰는 이유
- ☐ Forward mapping의 hole 문제, backward mapping 장점
- ☐ 보간 3종 (nearest/bilinear/bicubic) 차이
- ☐ Harris 구조 텐서 M 해석 (두 고유값)
- ☐ 좋은 특징 3조건 (불변성·변별성·재현성)
- ☐ SIFT vs ORB (L2 vs Hamming)
- ☐ Homography 자유도 8, 최소 대응점 4쌍
- ☐ Least Squares vs RANSAC
- ☐ 핀홀 투영식 + 주점/주축 정의
- ☐ 깊이 모호성 (α 배율로도 투영 동일)
- ☐ $Z = fB/d$ 스테레오 깊이식 + 수치 예시

15. 핵심 암기 카드 (시험 직전 1분)

숫자·공식 총정리

- **Homography**: 3×3 , 자유도 8, 최소 대응점 4쌍
- **Harris**: $R = \det(M) - k(\text{tr } M)^2$, 실습 $k=0.04$
- **핀홀**: $x = \frac{fX}{Z} + p_x$
- **스테레오**: $Z = \frac{fB}{d}$
- **Canny 4단계**: Gaussian $\rightarrow$ Gradient $\rightarrow$ NMS $\rightarrow$ Hysteresis
- **Opening**: Erosion $\rightarrow$ Dilation
- **Closing**: Dilation $\rightarrow$ Erosion
- **SIFT**: L2, 128D, 실수
- **ORB**: Hamming, 256 bits, 이진
- **PSNR**: $10 \log_{10}(\text{MAX}^2/\text{MSE})$
- **SSIM**: Luminance $\times$ Contrast $\times$ Structure

16. 마지막 한 줄

"영상의 의미 있는 점을 찾고, 그 점들의 기하 관계로 장면의 구조와 깊이를 복원한다"

— 이 문장 한 줄로 중간고사 전체 범위를 묶어 기억하면 된다.